Social Posting Automation Tool

Instagram + Facebook + LinkedIn + YouTube

Process, platform prerequisites, API flow, build phases, and master prompt

---

Purpose: Build a clean web application where a creator or business can upload content once, select platforms, edit captions per platform, and publish or schedule posts across Instagram, Facebook, LinkedIn, and YouTube.

Recommended approach: Use official APIs and OAuth. Do not collect user passwords, scrape sessions, or automate browser posting. Official APIs are slower to set up, but they are safer, scalable, and acceptable for real business use.

Best MVP rule: First make it work for your own accounts in developer/test mode. Then apply for platform approvals only after the UI, OAuth flow, and posting demo are working end-to-end.

One line product idea

A minimal black-and-white Apple-style dashboard that lets FounderLabs upload one media file, generate captions, select target platforms, click Post Now, and see live publish status for each platform.

1. What the application will do

- **Upload once:** User uploads image/video/document and enters a master caption.
- **Customize per platform:** The app creates platform-specific captions, hashtags, title, description, and privacy settings.
- **Connect accounts:** User connects Meta, LinkedIn, and Google/YouTube through OAuth.
- **Publish or schedule:** User can post immediately or schedule for later.
- **Track status:** Each platform shows pending, uploading, published, failed, retrying, or requires approval.
- **Store proof:** Save post IDs, published URLs, API response, error logs, and timestamps.

2. Recommended tech stack

Layer	Recommended choice	Why
Frontend	React + Vite + Tailwind CSS	Fast to build, clean UI, perfect for dashboard workflow.
Backend	FastAPI or Node.js/NestJS	FastAPI is simple for API + background jobs; Node has strong OAuth SDK support.
Database	PostgreSQL	Best for users, connected accounts, schedules, post logs, and audit records.
Queue	Redis + Celery/RQ/BullMQ	Posting should run in background so UI never freezes.
File storage	S3, Cloudflare R2, or Google Cloud Storage	Instagram/LinkedIn/YouTube often need public or uploadable media URLs.
Secrets	Cloud secret manager or encrypted DB columns	OAuth refresh tokens must be encrypted at rest.
AI captions	OpenAI or Gemini API optional	Generate platform-specific captions and hashtags.

3. High level architecture

1. Frontend uploads media and metadata to backend.
2. Backend stores the media in cloud storage and creates a post draft.
3. User connects each platform through OAuth. Tokens are encrypted and saved.
4. When user clicks Post Now, backend creates a job per selected platform.
5. Worker publishes to each official API and updates status in the database.
6. Frontend polls or subscribes through WebSocket/SSE to show live status.
7. Every API response, error, post ID, and URL is saved for debugging.

4. Common prerequisites before platform setup

- A domain name for production, for example app.yourdomain.com.
- A public privacy policy URL and terms URL. Most developer platforms ask for this.
- OAuth redirect URLs, for example https://app.yourdomain.com/api/oauth/meta/callback.
- Local callback URLs for development, for example http://localhost:8000/api/oauth/meta/callback.
- A database table for connected accounts and encrypted refresh/access tokens.
- A media storage bucket. For Instagram image/video publishing, the asset should be accessible to Meta during container creation.
- A small demo video/screencast of the app flow for app review submissions.
- A test account/page/channel for each platform. Do not test first on your main business accounts.

5. Platform prerequisites - summary

Platform	You need	Permissions / access	Main publish flow
Instagram	Professional Instagram account, Meta developer app, connected business assets, OAuth.	instagram_content_publish or current Instagram business content publishing permission; app review for external users.	Create media container, wait for processing, publish container.
Facebook	Facebook Page, Meta developer app, Page access token, Page admin/task access.	pages_show_list, pages_read_engagement, pages_manage_posts; publish_video for Page video uploads.	POST to Page feed, photos, or videos endpoint.
LinkedIn	LinkedIn developer app, OAuth, member profile or organization page access.	w_member_social for personal posting; organization posting needs approved organization/community management access.	Upload media if needed, then create post using LinkedIn Posts/UGC API.
YouTube	Google Cloud project, YouTube Data API v3 enabled, OAuth client, channel owner login.	youtube.upload or broader YouTube OAuth scope; quota and OAuth verification for production.	Upload via videos.insert with metadata and privacy settings.

Important: permissions and approval names change over time. When building, check the developer dashboard for the latest available scope names before submitting app review.

6. Instagram setup and publishing process

Prerequisites

- Meta Developer account and Meta app.
- Instagram Professional account. For the combined Meta flow, keep it linked to the Facebook Page you control.
- Business assets configured in Meta Business settings if you are posting for a brand.
- OAuth login enabled with correct redirect URI.
- Content publishing permission approved before posting for users outside your app roles/testers.
- Publicly accessible media URL or supported upload flow for the media object.

Publishing flow

8. User connects Instagram account through OAuth.
9. Backend fetches available Instagram business/professional account ID.
10. For image/video/Reel, backend creates an IG media container with media URL and caption.
11. Backend checks container processing status until it is ready.
12. Backend publishes the container using media_publish / creation_id.
13. Save Instagram media ID, permalink if available, status, timestamp, and API response.

Common mistakes to avoid

- Using a personal Instagram account instead of a professional account.
- Trying to publish before app review is approved for external users.
- Using a local file path instead of a public media URL or supported upload session.
- Skipping container status checks before publishing video/Reels.
- Requesting too many permissions during app review. Ask only for what your MVP actually uses.

7. Facebook Page setup and publishing process

Prerequisites

- Facebook Page where the authenticated user has the right Page tasks/role.
- Meta Developer app with Facebook Login.
- Page access token obtained from the authenticated user access token.
- pages_show_list to select Pages, pages_read_engagement to read Page info/status, pages_manage_posts to publish Page posts.
- publish_video permission if publishing videos to a Page.
- Privacy policy and app review screencast for production use.

Publishing flow

14. User connects Facebook through OAuth and grants Page permissions.
15. Backend lists Pages the user can manage and stores selected page_id.
16. Backend exchanges/uses Page access token and stores it securely.
17. For text/link post, call Page feed endpoint.
18. For photo post, call Page photos endpoint.
19. For video post, use the Page video upload flow.
20. Save page_post_id and show success/failure in dashboard.

8. LinkedIn setup and publishing process

Two LinkedIn posting modes

Mode	Best for MVP?	Requirement
Personal profile posting	Yes	Add Share on LinkedIn product and request w_member_social permission.
Company Page posting	Maybe later	Usually requires organization/community management access and admin relationship to the Page. Approval can take time.

Prerequisites

- LinkedIn Developer app.
- Verified app/company association if required by LinkedIn.
- OAuth 2.0 redirect URL configured.
- w_member_social for posting on behalf of an authenticated member.
- For organization posting, the user must be an admin and the app may require approved Community Management/Marketing API access.
- Use current LinkedIn-Version header for REST APIs.

Publishing flow

21. User connects LinkedIn through OAuth.
22. Backend stores encrypted access/refresh token if issued.
23. For text post, create a LinkedIn post with author URN and commentary.
24. For image/video/document post, upload asset first and get media URN.
25. Create post referencing that media URN.
26. Save LinkedIn post URN, status, and response headers.

Practical suggestion: Start with personal LinkedIn posting first. Add company page posting after you have approval and working organization URN discovery.

9. YouTube setup and publishing process

Prerequisites

- Google Cloud project.
- YouTube Data API v3 enabled.
- OAuth consent screen configured with app name, support email, authorized domain, privacy policy, and scopes.
- OAuth 2.0 client ID and client secret for web application.
- Redirect URI configured for your backend callback.
- Channel owner must connect their YouTube account through OAuth.
- youtube.upload scope or broader YouTube scope for uploading videos.
- Check upload quota in Google Cloud before production use.

Publishing flow

27. User connects Google/YouTube through OAuth.
28. Backend stores encrypted refresh token.
29. User uploads video and enters title, description, tags, category, privacy status, and optional schedule time.
30. Backend starts resumable upload using videos.insert.
31. Backend tracks upload progress and updates UI.
32. Save YouTube video ID, watch URL, status, and upload response.

10. Minimal database design

Table	Important columns
users	id, name, email, password_hash or auth_provider, created_at
connected_accounts	id, user_id, platform, account_name, external_account_id, encrypted_access_token, encrypted_refresh_token, expires_at, scopes, status
media_assets	id, user_id, file_url, file_type, file_size, duration, thumbnail_url, created_at

post_drafts	id, user_id, title, master_caption, status, scheduled_at, created_at
post_targets	id, post_draft_id, platform, account_id, caption, title, description, privacy, status
publish_jobs	id, post_target_id, job_status, attempts, last_error, external_post_id, external_url, api_response_json, created_at, updated_at
audit_logs	id, user_id, action, platform, request_id, metadata_json, created_at

11. Backend API endpoints to build

Endpoint	Purpose
POST /api/media/upload	Upload image/video and store in cloud storage.
POST /api/posts/draft	Create post draft with master caption and media asset.
PATCH /api/posts/{id}	Edit post draft and platform-specific copy.
POST /api/oauth/meta/start	Start Instagram/Facebook OAuth.
GET /api/oauth/meta/callback	Complete Meta OAuth and store tokens.
POST /api/oauth/linkedin/start	Start LinkedIn OAuth.
GET /api/oauth/linkedin/callback	Complete LinkedIn OAuth and store tokens.
POST /api/oauth/google/start	Start Google/YouTube OAuth.
GET /api/oauth/google/callback	Complete Google OAuth and store tokens.
POST /api/posts/{id}/publish	Create one publish job per selected platform.
GET /api/posts/{id}/status	Return live status of each target.
POST /api/jobs/{id}/retry	Retry failed posting job.

12. Frontend screens

- Login / workspace screen.
- Connect Accounts screen: Meta, LinkedIn, YouTube connection cards.
- Create Post screen: upload media, write master caption, AI caption button, select platforms.
- Platform Preview screen: edit Instagram caption, Facebook caption, LinkedIn text, YouTube title/description/privacy.
- Publish Confirmation modal: show selected accounts and warnings.
- Live Status screen: one card per platform with progress, success URL, or error message.
- Post History screen: previous posts, filters, retry buttons, API response logs for admin.
- Settings screen: privacy policy URL, timezone, default hashtags, brand tone, token reconnect.

13. Build phases

Phase	Goal	Output
Phase 1	Frontend demo only	Upload UI, platform selector, loader, fake success status.
Phase 2	Backend and database	Real media upload, post drafts, history, job table.
Phase 3	OAuth connections	Connect Meta, LinkedIn, Google accounts and save encrypted tokens.
Phase 4	Real posting - one platform	Start with Facebook Page or LinkedIn personal profile.
Phase 5	Add remaining platforms	Instagram container publish, YouTube upload, LinkedIn media upload.
Phase 6	Scheduling and retries	Queue worker, retry logic, scheduled publishing.
Phase 7	App review package	Privacy policy, screencast, test user, permission explanation.

14. Master prompt for Antigravity, Cursor, Claude Code, or similar builder

Copy and paste this prompt into your coding tool. Replace bracketed values before running it.

```
You are a senior full-stack engineer and product designer. Build a production-ready MVP called FounderLabs Social Auto Poster.

Goal:
Create a clean black-and-white Apple-style web app that lets a user upload content once, select Instagram, Facebook, LinkedIn, and YouTube, customize captions per platform, then publish immediately or schedule posts.

Important rules:
- Use official APIs only. Do not collect platform passwords. Do not automate browser sessions.
- OAuth tokens must be encrypted at rest.
- Posting must run in background jobs, not inside the frontend request.
- Every platform publish attempt must create a log with status, external post ID, URL, raw API response, and error message.

Tech stack:
- Frontend: React + Vite + Tailwind CSS.
```

```
- Backend: FastAPI with Python 3.12.
- Database: PostgreSQL using SQLAlchemy or SQLModel.
- Queue: Redis + Celery or RQ.
- Storage: local storage for dev, S3/R2/GCS abstraction for production.
- Auth: simple email/password or magic link for MVP.

Frontend requirements:
1. Landing/dashboard page with title: FounderLabs Social Auto Poster.
2. Connect Accounts page with cards for Meta, LinkedIn, and YouTube.
3. Create Post page:
   - Drag and drop media upload.
   - Master caption input.
   - Platform checkboxes: Instagram, Facebook, LinkedIn, YouTube.
   - Generate platform captions button using placeholder function for now.
   - Preview cards for each selected platform.
   - Post Now and Schedule buttons.
4. Live Status page:
   - Show each platform as a status card: Pending, Uploading, Published, Failed, Retry.
   - Show external URL after success.
5. Post History page with filters by platform and status.

Backend requirements:
1. Models:
   - User
   - ConnectedAccount
   - MediaAsset
   - PostDraft
   - PostTarget
   - PublishJob
   - AuditLog
2. APIs:
   - POST /api/media/upload
   - POST /api/posts/draft
   - PATCH /api/posts/{id}
   - POST /api/posts/{id}/publish
   - GET /api/posts/{id}/status
   - POST /api/jobs/{id}/retry
   - POST /api/oauth/meta/start
   - GET /api/oauth/meta/callback
   - POST /api/oauth/linkedin/start
   - GET /api/oauth/linkedin/callback
   - POST /api/oauth/google/start
   - GET /api/oauth/google/callback
3. Create platform service classes:
   - MetaFacebookPublisher
   - InstagramPublisher
   - LinkedInPublisher
   - YouTubePublisher
4. For the first working version, implement mock publishers that return fake external URLs.
5. Then add real API integrations behind feature flags:
   - ENABLE_FACEBOOK_REAL=true
   - ENABLE_INSTAGRAM_REAL=true
   - ENABLE_LINKEDIN_REAL=true
   - ENABLE_YOUTUBE_REAL=true

Platform logic:
- Facebook: publish Page text/photo/video posts using Page access token.
- Instagram: create media container, poll processing status, publish media container.
- LinkedIn: create member post first; support organization posting later after approval.
- YouTube: use resumable videos.insert upload with title, description, tags, privacy status.

Environment variables:
DATABASE_URL=
REDIS_URL=
```

```
APP_BASE_URL=
ENCRYPTION_KEY=
META_APP_ID=
META_APP_SECRET=
META_REDIRECT_URI=
LINKEDIN_CLIENT_ID=
LINKEDIN_CLIENT_SECRET=
LINKEDIN_REDIRECT_URI=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=
STORAGE_PROVIDER=local
STORAGE_BUCKET=

Deliverables:
- Complete project folder structure.
- README with setup commands.
- .env.example file.
- Database migrations.
- Mock publisher demo working end-to-end.
- Clear TODO comments where real API keys and app review approvals are needed.
- UI should be minimal, premium, spacious, black-and-white, and responsive.
```

15. App review and production checklist

- Make the app reachable on a public HTTPS domain before review.
- Create test credentials and write exact reviewer instructions.
- Record a clear screencast: login, connect platform, create post, publish, show result on platform.
- Show exactly why each permission is needed. Do not request future permissions.
- Add privacy policy, terms, and data deletion instructions.
- Add reconnect flow for expired tokens.
- Add rate limit handling and retry backoff.
- Add audit logs for every publish action.
- Add manual approval before publishing to avoid accidental posts.
- Add per-platform validation: video duration, aspect ratio, title length, caption length, file type, and file size.

16. Development notes for a clean MVP

- Start with fake/mock posting so the UI demo works immediately.
- Then integrate one real platform at a time. Facebook Page posting is usually the easiest Meta starting point.
- Do not build all real APIs in one shot. OAuth debugging becomes painful if every platform fails at once.
- Keep platform-specific caption fields separate. A YouTube title is not the same as an Instagram caption.
- Store all external post IDs. Without them, you cannot debug, retry, update, or show proof of posting.
- Never expose access tokens in frontend logs, network responses, screenshots, or browser storage.
- For FounderLabs sessions, use the mock version first, then show one real platform live to avoid API review delays blocking the demo.

17. Sources checked while preparing this guide

- Meta - Instagram Content Publishing: https://developers.facebook.com/docs/instagram-platform/content-publishing/
- Meta - Instagram media publish reference: https://developers.facebook.com/docs/instagram-platform/instagram-graph-api/reference/ig-user/media_publish/
- Meta - Pages API getting started: https://developers.facebook.com/docs/pages-api/getting-started/
- Meta - Pages API posts: https://developers.facebook.com/docs/pages-api/posts/
- Meta - Permissions reference: https://developers.facebook.com/docs/permissions/
- LinkedIn - Getting Access to LinkedIn APIs: https://learn.microsoft.com/en-us/linkedin/shared/authentication/getting-access
- LinkedIn - Share on LinkedIn: https://learn.microsoft.com/en-us/linkedin/consumer/integrations/self-serve/share-on-linkedin
- LinkedIn - Posts API: https://learn.microsoft.com/en-us/linkedin/marketing/community-management/shares/posts-api
- YouTube Data API reference: https://developers.google.com/youtube/v3/docs
- YouTube Upload a Video guide: https://developers.google.com/youtube/v3/guides/uploading_a_video
- YouTube videos.insert reference: https://developers.google.com/youtube/v3/docs/videos/insert